

TwinaForms - Describe Your Solution

Date: 2026-06-05

Package version: TwinaForms managed package 0.10.0.5

Subscriber package version id: 04tgL000000GburQAC

Executive Summary

TwinaForms is a managed Salesforce package and AWS-hosted runtime that lets Salesforce admins design, publish, and operate Salesforce-connected web forms.

The Salesforce package provides the admin setup flow to authenticate the subscriber org with TwinaForms through Salesforce OAuth, then design and publish forms to AWS.

The AWS runtime hosts public form artifacts, validates published form requests, performs tenant-scoped Salesforce API operations, stores runtime security records, and exposes protected service APIs used by the package.

The solution is designed for multi-tenant packaged use. Each subscriber Salesforce org is treated as a separate tenant identified by its Salesforce orgId. The security model intentionally separates subscriber org connection trust, package-to-AWS service trust, and public form execution trust. After the customer connects their Salesforce org with OAuth, TwinaForms creates a private per-org secret that lets Salesforce prove to AWS that later package service calls, such as connection verification and form publishing, really came from the installed TwinaForms package in that org.

When a form is published, the Salesforce package generates a per-form publish token. AWS stores a hashed copy with the server-side form policy, and the published form uses that token when calling the prefill and submit endpoints. The public browser never receives the tenant service secret, Salesforce OAuth refresh token, AWS secrets, or executable Salesforce command definitions.

Solution Components

Salesforce Managed Package

The TwinaForms Salesforce package includes the following main component groups:

- Lightning Web Components for Connect, Home, Designer, published-form management, submission logs, prefill/submit configuration, permission management, and setup workflows.
- Apex controllers for setup, publishing, AWS callouts, tenant entitlement checks, access management, demo data, user verification, and submission log handling.
- Custom objects and metadata that store form definitions, form versions, form elements, form actions, publication records, project organization, user-facing configuration, and submission-log cryptography metadata.
- Permission sets for customer users and package setup access.

- Remote Site Settings for direct HTTPS callouts from packaged Apex to the TwinaForms AWS endpoints.

The Salesforce package is the subscriber-facing administration surface. It is responsible for building forms, configuring allowed Salesforce objects/fields/actions, publishing generated HTML artifacts, and calling AWS service endpoints through package-controlled callout logic.

AWS Runtime

The AWS runtime contains these main Lambda services:

- Salesforce backend Lambda for tenant setup, OAuth callback handling, tenant status, signed service-access checks, form registration, publish/unpublish support, plan/entitlement responses, and Salesforce metadata assist routes.
- Prefill Lambda for public browser prefill requests against published form policy.
- Submit Lambda for public browser submit requests against published form policy.
- Submission Logs API Lambda for protected Salesforce-facing submission log list/detail/config routes.

AWS storage includes:

- DynamoDB tenant records keyed by Salesforce orgId.
- DynamoDB form security records keyed by form id and associated with the owning orgId.
- DynamoDB submission log records with tenant-scoped indexes and TTL retention.
- AWS Secrets Manager entries for the central TwinaForms OAuth client configuration and per-org Salesforce connection records.

Public form HTML is hosted separately from Lambda services. Runtime APIs are HTTPS endpoints and are validated server-side before any Salesforce read/write operation is attempted.

Setup and Trust Flow

Tenant Registration and OAuth

When a subscriber installs TwinaForms, an admin opens the packaged Connect page and prepares the org connection. The Connect page gathers org context such as orgId, admin email, company name, and the org login base URL. The admin then authorizes TwinaForms through Salesforce OAuth.

AWS exchanges the Salesforce OAuth authorization code server-to-server using TwinaForms-owned OAuth client credentials. AWS validates that the Salesforce org returned by OAuth matches the org that initiated setup. AWS then stores the org-specific refresh token, instance URL, and connection metadata in AWS Secrets Manager under that org id.

Subscriber admins do not create Salesforce Named Credentials, External Credentials, or customer-managed OAuth client secrets for this setup path.

Package Service Access

After OAuth succeeds, AWS uses the freshly authorized Salesforce OAuth access token to call a packaged Apex endpoint inside the authenticated subscriber org. Salesforce requires the OAuth access token, and

Apex validates that the requested orgId matches `UserInfo.getOrganizationId()` before returning or creating the private per-org signing secret. AWS stores the resulting signing secret in the org-specific Salesforce connection secret.

Later package-to-AWS calls are signed by Apex with HMAC headers. Each signed request includes the org id, timestamp, nonce, body hash, and signature. The 5-minute validation window applies to each later signed package-to-AWS request, not to the initial OAuth-time secret retrieval. AWS accepts an individual signed request only within that timestamp window and records each nonce so the same signed request cannot be replayed successfully after it has already been used.

This protects package service calls without exposing the signing secret to the browser or requiring manual subscriber credential setup.

Protected package-to-AWS calls include tenant service-health checks, form registration, publish/unpublish operations, tenant entitlement/home-summary calls, Salesforce metadata assist routes, and submission log API calls.

Public Form Runtime Security

TwinaForms separates tenant trust from form trust.

Tenant trust is used for package-originated service operations from Salesforce to AWS. Form trust is used for public browser prefill and submit operations.

Each published form has:

- a formId
- an owning Salesforce orgId
- a published version id
- a per-form publish token
- a server-side form security record in DynamoDB
- prefill and submit policies
- stored prefill and submit definitions generated by the Salesforce package

The public browser sends only the form id, publish token, and user input or lookup parameters. The browser does not send executable command definitions, object allowlists, field allowlists, tenant secrets, OAuth tokens, or Salesforce credentials.

For prefill, AWS validates:

- the form id exists
- the publish token hash matches the stored token hash
- the form status is published
- the owning tenant is active
- the prefill command types are allowed

- the configured Salesforce objects are allowed
- request parameters match the server-side policy

For submit, AWS validates:

- the form id exists
- the publish token hash matches the stored token hash
- the form status is published
- the owning tenant is active
- the submit command types are allowed
- the configured Salesforce objects are allowed
- writable fields are allowed by policy
- higher-risk commands such as update/delete/upsert are allowed only by the configured security mode and policy

This means the browser cannot choose arbitrary Salesforce objects, fields, SOQL fragments, or write operations at request time. Runtime Salesforce reads and writes are driven by the server-side definitions created during publish.

Salesforce Data Access

When AWS needs to call Salesforce for prefill, submit, layout metadata, or other runtime operations, AWS resolves the tenant from the form or request context, loads the org-specific Salesforce connection, refreshes the access token server-side, and calls the subscriber org. The central TwinaForms OAuth client secret is stored in AWS and is not exposed to Salesforce users or public browsers.

Data access is tenant-scoped by orgId. Form security records include the owning org id, and runtime services load the tenant record before proceeding. Public requests cannot use one tenant's publish token to access another tenant's form because form id, token hash, tenant ownership, form status, and policy are checked together.

Submission Logs

TwinaForms records operational submission history for subscriber-org auditability and troubleshooting inside Salesforce. Submission logs are tenant-scoped and use plan-based retention. Basic metadata records outcome, submission reference, form id, time, and failure category.

Detailed submission log payloads are encrypted before storage when the tenant plan supports detail logs and the tenant key configuration is ready. The design uses a hybrid encryption model:

- Salesforce stores hidden org-level cryptographic material.
- AWS stores only the synced public key and key version.
- AWS encrypts detail payloads with a one-time AES key.

- AWS encrypts the AES key with the org public key.
- The Salesforce Submission Logs UI decrypts detail using the org-side private key.

Plain metadata does not include raw Salesforce/API failure text or submitted field values. Those details are stored only in the encrypted detail payload when enabled.

Security Controls Summary

Key controls include:

- Salesforce OAuth authorization code flow with refresh token support.
- Org id validation during OAuth callback.
- One AWS Secrets Manager connection record per subscriber org.
- HMAC signing for package-to-AWS service calls using the private per-org signing secret.
- Protected package storage for org signing secrets.
- Per-form publish tokens generated by the Salesforce package and stored as hashes in AWS.
- Server-side form policies for command, object, and field allowlists.
- Tenant-scoped DynamoDB records and tenant-active checks.
- Separation of tenant service secrets from public form publish tokens.
- No browser-visible tenant secret, OAuth refresh token, AWS secret, or executable command definition.
- Structured error responses and customer-safe UI messages.
- Submission detail encryption before DynamoDB storage.

Security Testing Evidence

The current ISV evidence packet is stored in:

- security-reports/isv/TwinaForms_ISV_Security_Review_Packet_2026-06-05.md
- security-reports/code-analyzer/isv-submission-2026-06-05
- security-reports/zap/isv-submission-2026-06-05

Salesforce Code Analyzer focused AppExchange scan results:

- 28 moderate findings.
- No high or critical findings in the focused AppExchange scan.
- Findings are documented as false positives or accepted public values related to signature, token, and key terminology.

OWASP ZAP results:

- Six passive requestor scan plans succeeded.
- Protected Salesforce backend and submission-log routes reject unauthenticated requests.

- Public prefill and submit endpoints enforce publish-token and form-policy validation.
- Known public-form browser-header and AWS-managed endpoint findings are documented in the ZAP response notes.

Related Documentation Included for Review

The following documentation files are prepared for reviewer upload or reference:

- TwinaForms_Describe_Your_Solution_2026-06-05.pdf
- TwinaForms_Describe_Your_Solution_2026-06-05.docx
- TwinaForms_Describe_Your_Solution_2026-06-05.md
- security-reports/isv/TwinaForms_ISV_Security_Review_Packet_2026-06-05.md
- security-reports/code-analyzer/isv-submission-2026-06-05/TwinaForms_Code_Analyzer_ISV_Response_2026-06-05.md
- security-reports/zap/isv-submission-2026-06-05/responses/TwinaForms_ZAP_ISV_Response_2026-06-05.md
- security-reports/zap/isv-submission-2026-06-05/responses/TwinaForms_ZAP_Scope_Note_2026-06-05.md